DATA305 / DATA475 · Machine Learning Methods

Image Segmentation

Pixel-wise Prediction · U-Net · SAM

What and exactly where — at every pixel

Lectures 11-12

Week 9 · Google Colab ready

Binh Nguyen · School of Mathematics and Statistics · Victoria University of Wellington

Today's Roadmap & Learning Objectives

Five sections · Oxford-IIIT Pet · U-Net architecture

01

What is segmentation?

Tasks, masks, and why it's harder than classification.

02

Masks, outputs, metrics

Pixel-wise classification, IoU and Dice.

03

FCN and U-Net

Encoder, decoder, and skip connections.

04

Keras demo

U-Net on Oxford-IIIT Pet, with metrics.

05

Beyond U-Net

Instance masks, YOLO-seg, and SAM.

Learning Objectives

✓ Distinguish classification, detection, and the segmentation tasks.

✓ Explain U-Net: encoder, decoder, skip connections.

✓ Evaluate segmentation with IoU, not just accuracy.

Disclosure: Claude was used to support organization, templating, coding, and formatting; all lecture ideas, content, adaptation, final decisions, and academic responsibility remain with the lecturer.

SECTION 01

What is segmentation?

01

Tasks, masks, and why it's harder than classification.

From Classification to Segmentation

Section 01

Three computer-vision tasks — same image, different output structure.

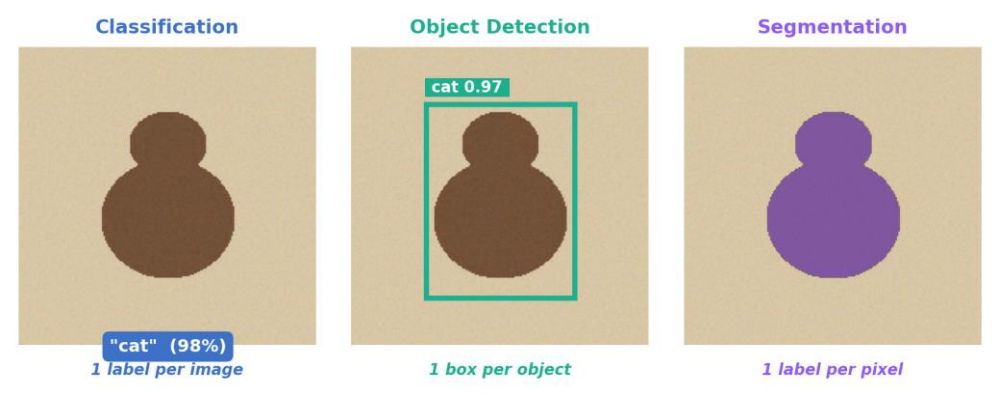
Each task answers a stricter question:

→ Classification — what is in the image?

→ Detection — what and roughly where, using bounding boxes.

→ Segmentation — what and exactly where, at every pixel.

Output complexity grows from 1 label → ~10 boxes → 100,000s of pixel labels.



Semantic, Instance, and Panoptic Segmentation

Section 01

Three flavors of pixel-wise prediction.

Semantic

Every pixel gets a class label.

- All cats share one mask.
- No notion of object identity.
- Output: H × W integer map.

Today's main focus.

Instance

Every object has its own mask.

- Cat #1 ≠ Cat #2 even if same class.
- Built on top of detection.
- Output: list of (class, mask) pairs.

Mask R-CNN, YOLO-seg.

Panoptic

Semantic + instance, unified.

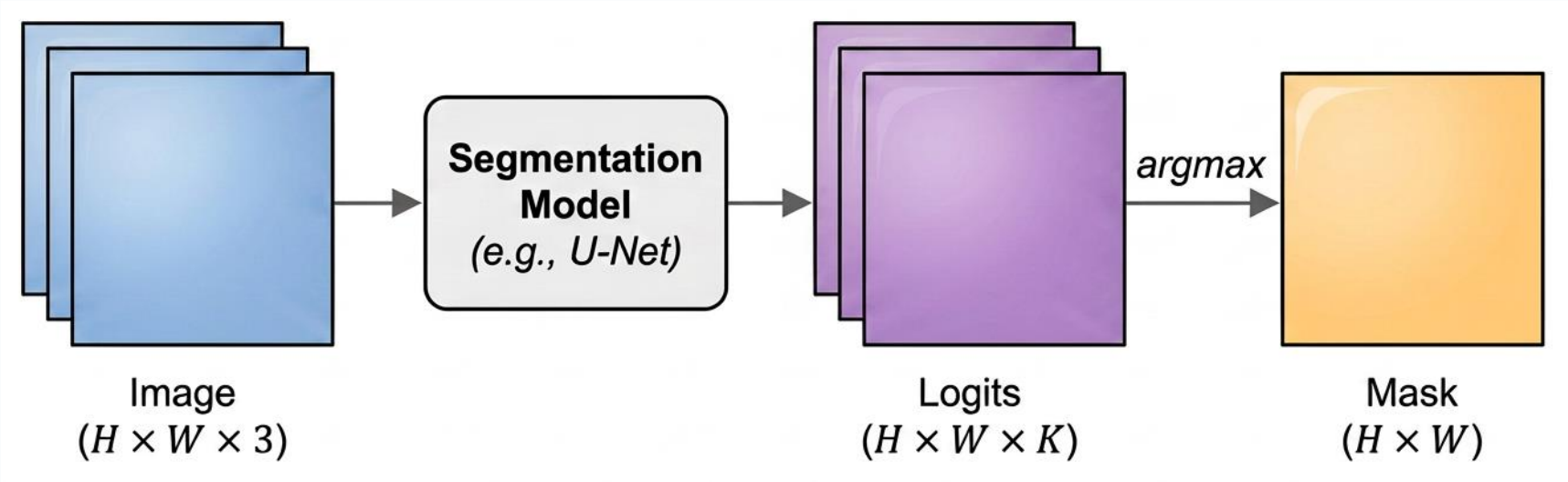
- Things (countable) get instance IDs.
- Stuff (sky, road) is just labeled.
- Every pixel is assigned exactly once.

Combines both views.

Mind the difference: instance separates objects of the same class; panoptic covers everything

Semantic Segmentation — A Label for Every Pixel

Input H × W × 3 → output H × W of integer class labels.



Key idea — semantic segmentation is pixel-wise classification: the model assigns one class label to every pixel in the image.

Spatial structure must be preserved end-to-end — we cannot flatten away rows and columns.

Output is a H × W integer mask; for K classes, model outputs H × W × K logits

Why Segmentation Is Hard

Section 01

Two opposing demands: global context and pixel-precise boundaries.

Global context

The model must know what the object IS to label it correctly.

Local precision

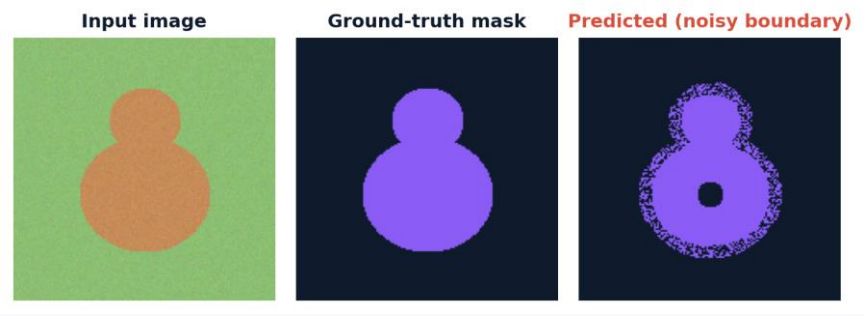
Boundaries are 1–2 pixels wide — small errors are very visible.

Class imbalance

Background pixels can dominate; small objects vanish in metrics.

Boundary ambiguity

Hair, fur, or shadows blur the edge between object and background.



Many pixel errors live near boundaries.

These four pressures shape architecture choices and metrics used in segmentation

Quick Quiz — Which Segmentation Task?

Section 01

For each scenario, decide: semantic, instance, or panoptic?

01

All road pixels labeled "road", all cars labeled "car" — but cars are not separated.

?

02

Each car gets its own mask with a unique ID; the road has no instances.

?

03

Every single pixel has either a class (sky, road) or an instance ID (car #3) — never both, never neither.

?

Discuss with your neighbor — answer on the next slide

Answer — Which Segmentation Task?

Section 01

Semantic labels stuff; instance separates things; panoptic does both at once.

01

All pixels get class labels; identical cars share a label and are not separated.

→ **Semantic**

02

Each car has its own object mask; "stuff" classes like road may be ignored.

→ **Instance**

03

Every pixel is assigned exactly once: semantic labels for stuff, instance IDs for things.

→ **Panoptic**

Each task answers a different question about the same image.

SECTION 02

Masks, outputs, metrics

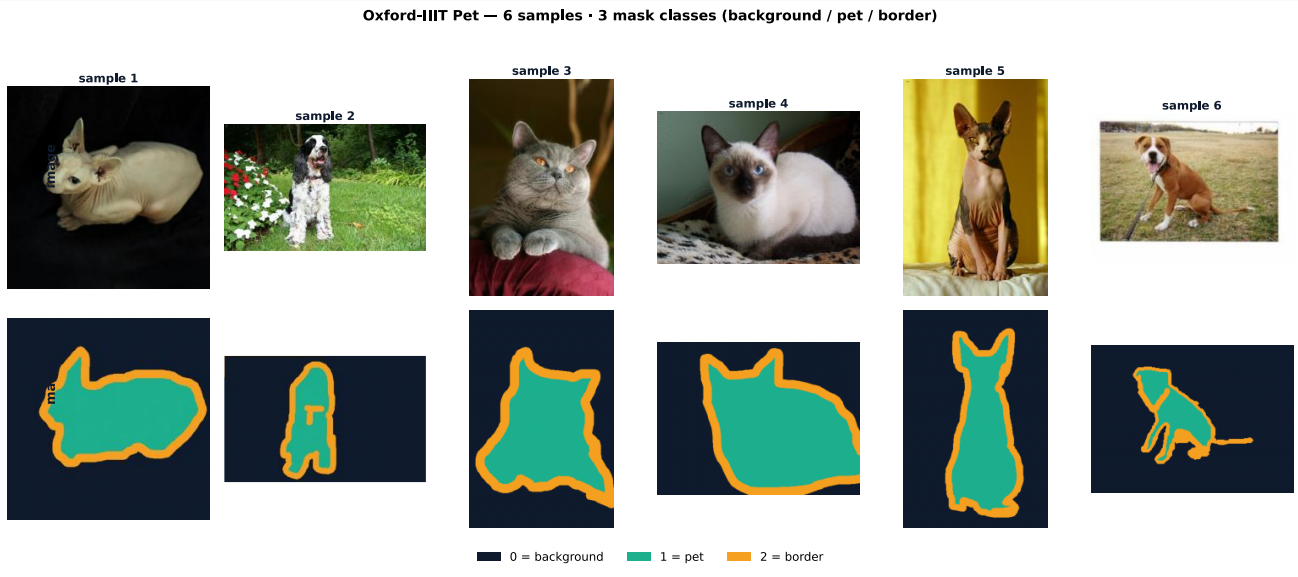
02

Pixel-wise classification, IoU and Dice.

What Does a Segmentation Mask Look Like?

Section 02

An integer image, one entry per pixel.



Oxford-IIIT Pet

Each mask uses three integer values:

0 = background

1 = pet

2 = border

The grayscale view is hard to read — color-mapping is purely for human inspection.

Mask shape: (H, W) of integer dtype · same H, W as the image, no channel axis.

Convention varies — some datasets use (H, W, 1) or one-hot (H, W, K)

Quick Quiz — Resize Mode for Each Tensor

Section 02

For each tensor, choose the interpolation: bilinear, nearest, or both work.

01

You resize the RGB photo from 500×400 down to 128×128.

?

02

You resize the segmentation mask (class IDs 0, 1, 2) the same way.

?

03

You upsample the model's softmax probabilities back to the original size.

?

Answer — Resize Mode for Each Tensor

Section 02

Use bilinear when values are continuous; use nearest when values are class IDs.

01

You resize the RGB photo from 500×400 down to 128×128.

→ **Bilinear**

02

You resize the segmentation mask (class IDs 0, 1, 2) the same way.

→ **Nearest only**

03

You upsample the model's softmax probabilities back to the original size.

→ **Bilinear**

Bilinear creates fractional class IDs — never use it on masks. If resizing an argmax mask, use nearest.

Why Mask Resizing Can Break Your Labels

Section 02

Images are continuous; segmentation masks are discrete class IDs.

RGB image

Continuous intensities

Bilinear interpolation is fine

Smoother downsampling

Segmentation mask

Integer class IDs

Bilinear creates invalid labels

Use nearest-neighbor only

- Fractional class IDs are not new classes — they are preprocessing bugs.
- The same spatial transform must be applied to the image and its mask.

This is one of the most common bugs in segmentation notebooks.

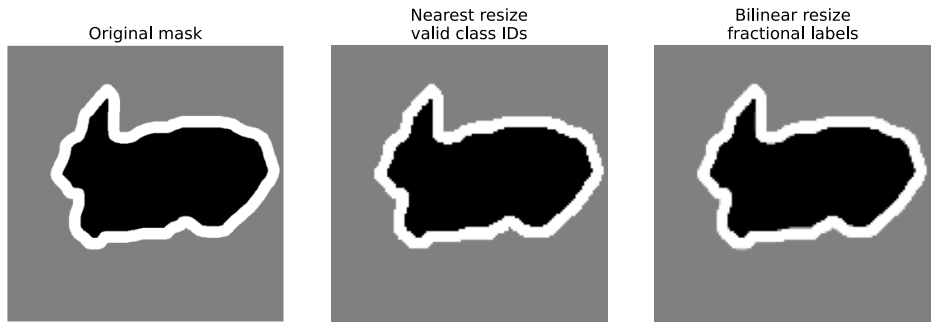
Code Demo — Nearest vs Bilinear on a Mask

Section 02

The printed unique values reveal the bug immediately.

```
mask_nearest  = tf.image.resize(raw_mask, [128,128],
method="nearest")
mask_bilinear = tf.image.resize(raw_mask, [128,128],
method="bilinear")

print(np.unique(mask_nearest.numpy().astype(int)))
print(np.unique(np.round(mask_bilinear.numpy().ravel(),
3))[:20])
```



Nearest output

[1, 2, 3]

Valid class IDs only

Bilinear output

[1.000, 1.031, 1.250, ...]

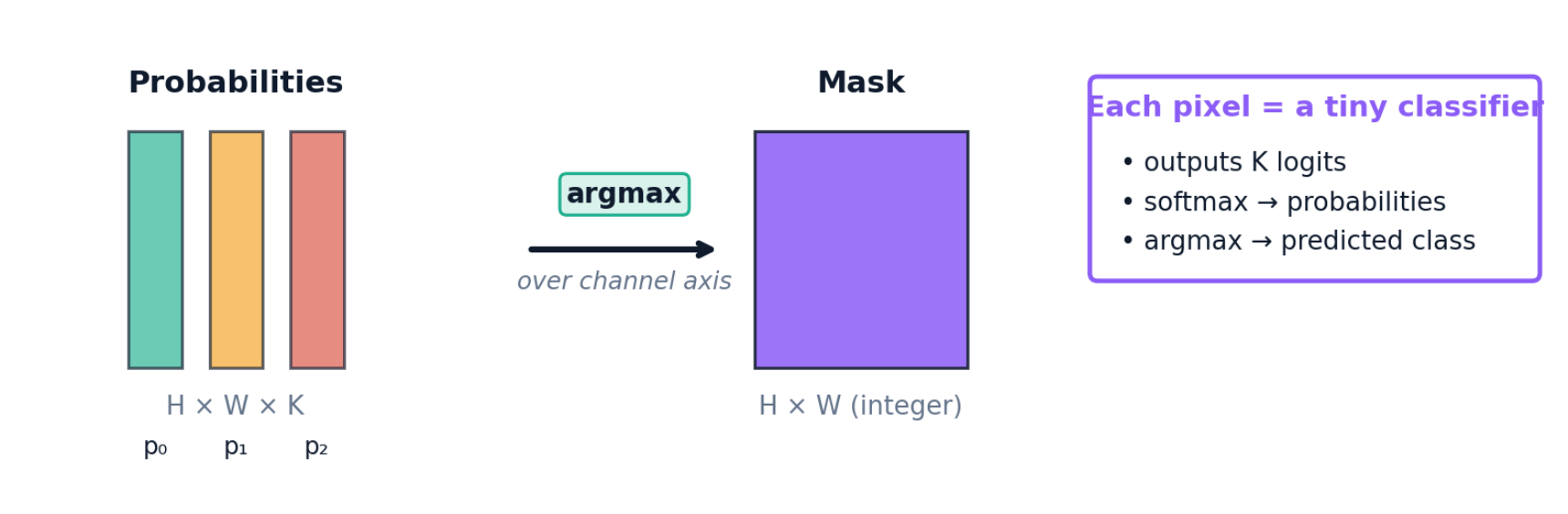
Invalid fractional labels

Rule

Use bilinear for image pixels; use nearest for masks after labels are assigned.

Pixel-wise Classification

Each pixel runs a tiny classifier; the mask is just argmax.



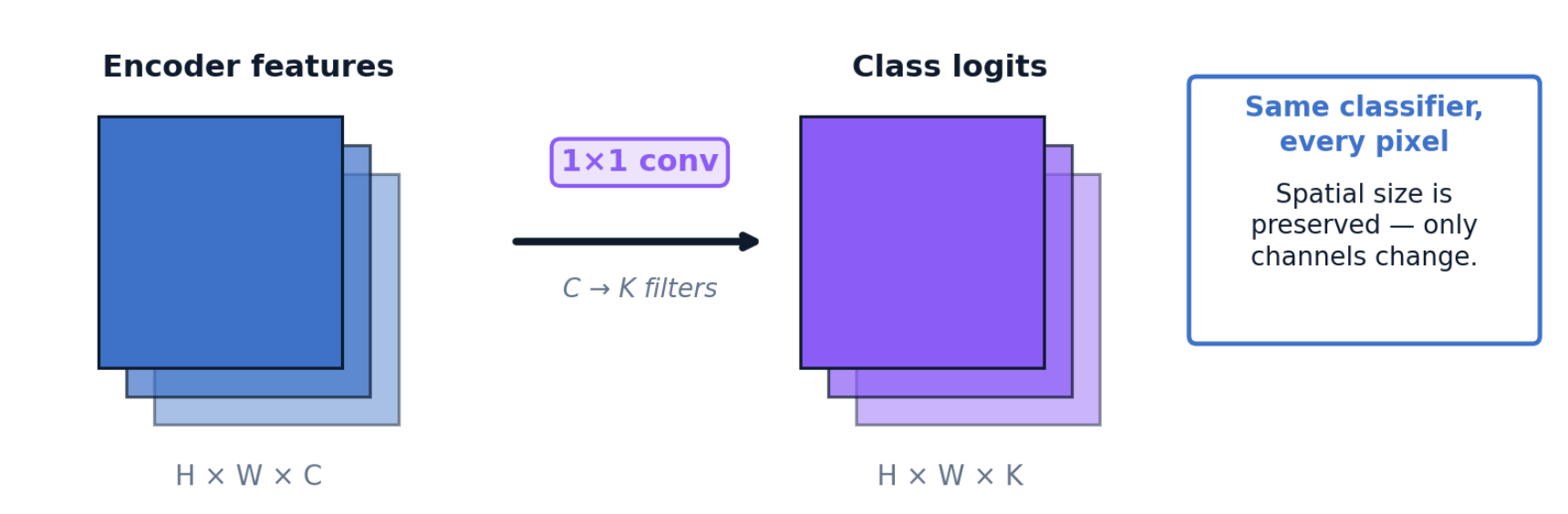
- Treat segmentation as H × W parallel classifiers, one per pixel position.
- All pixel-classifiers share the same weights (no overfitting per location).
- Loss is computed and averaged across every pixel of every training image.

Conceptually: same softmax classifier, applied identically at every spatial position

The 1×1 Convolution Output Layer

Section 02

How a feature vector at each pixel becomes K class logits.



Why 1×1? A 1×1 convolution acts like the same small classifier applied at every spatial location.

It keeps the spatial layout of the feature map and only changes the channel count: C → K.

In Keras: layers.Conv2D(K, 1, activation="softmax") on the final feature map

Loss Functions for Segmentation

Section 02

Pixel-wise cross-entropy is the workhorse — but mind class imbalance.

Multi-class masks

Sparse categorical cross-entropy

Targets are integer class IDs (0, 1, 2, …); model outputs a softmax over K classes per pixel.

loss = - log p(true_class) averaged over every pixel of every image.

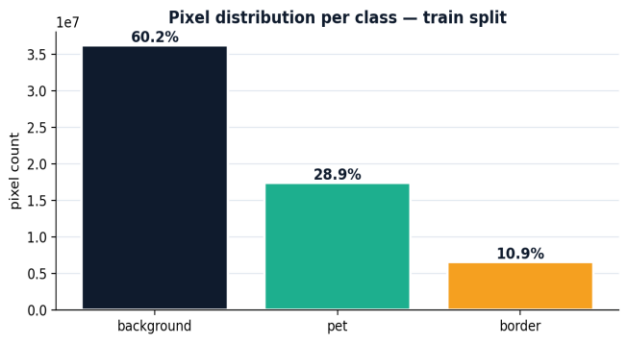
Used in our Oxford Pet U-Net.

Foreground / background

Binary cross-entropy

Targets are 0/1 masks; model outputs a single sigmoid per pixel.

Equivalent to K=2 cross-entropy with a single output channel.



⚠ **Class imbalance** — background is the largest class; in this run it is about **60.2%** of pixels. The loss can collapse onto background.

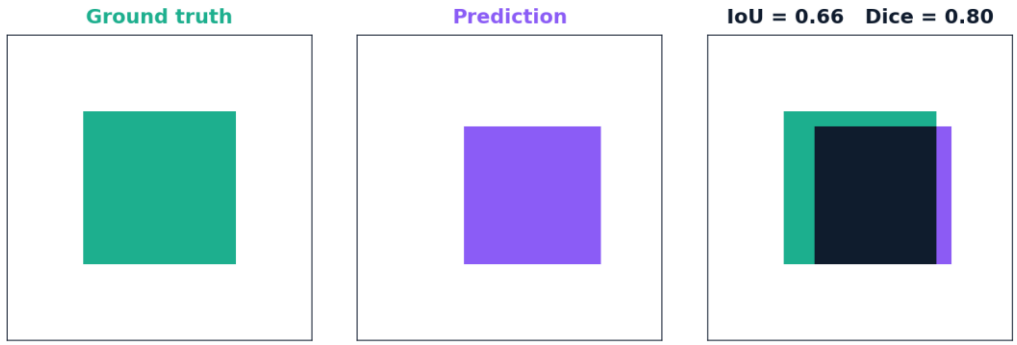
Mitigations: class weights in the loss, sample weighting, focal loss, or Dice loss.

Keras: loss="sparse_categorical_crossentropy" for integer targets, "binary_crossentropy" for 0/1

Evaluation Metrics — Pixel Accuracy, IoU, and Dice

Section 02

Accuracy can mislead; IoU and Dice focus on object overlap.



$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|} \quad \cdot \quad \text{Dice} = \frac{2|A \cap B|}{|A| + |B|}$$

Pixel accuracy

Fraction of pixels labeled correctly. Easy to compute, but biased toward majority class.

Intersection over Union (IoU)

$|A \cap B| / |A \cup B|$. Penalizes both false positives and false negatives — the standard metric.

Dice coefficient

$2|A \cap B| / (|A| + |B|)$. Equivalent to F1 for masks; more sensitive to small objects.

Keras provides tf.keras.metrics.MeanIoU; Dice is usually defined manually

Quick Quiz — When Accuracy Misleads

Section 02

An image where 95% of pixels are background. The model predicts "all background."

Scenario

Image of size **100 × 100** = 10,000 pixels.

True positives (object): **500**

True background: **9,500**

Prediction: **every pixel = background**

What does each metric report?

Your job

1. **Compute pixel accuracy.**

   Hint: count the pixels that match the prediction.

2. **Compute IoU on the object class.**

   Hint: how many object pixels did the model recover?

Which metric tells the truth?

Answer — When Accuracy Misleads

Section 02

The same scenario — now scoring the all-background prediction with both metrics.

Scenario

Image of size **100 × 100** = 10,000 pixels.

True positives (object): **500**

True background: **9,500**

Prediction: **every pixel = background**

What does each metric report?

Results

Pixel accuracy = 9,500 / 10,000 **= 0.95**

Looks excellent — but the object is invisible.

IoU (object class) = 0 / 500 **= 0.00**

Tells the truth — model is useless on the object.

→ **Pixel accuracy looks great here; per-class IoU exposes the failure.**

This is why segmentation models should report IoU or Dice, not accuracy alone.

IoU from a Confusion Matrix

Section 02

For each class, IoU counts overlap while penalizing both false positives and false negatives.

True positive
Pixels correctly predicted as class c.

False positive
Pixels predicted as class c but actually another class.

False negative
Pixels of class c predicted as another class.

$$IoU_c = TP_c / (TP_c + FP_c + FN_c)$$

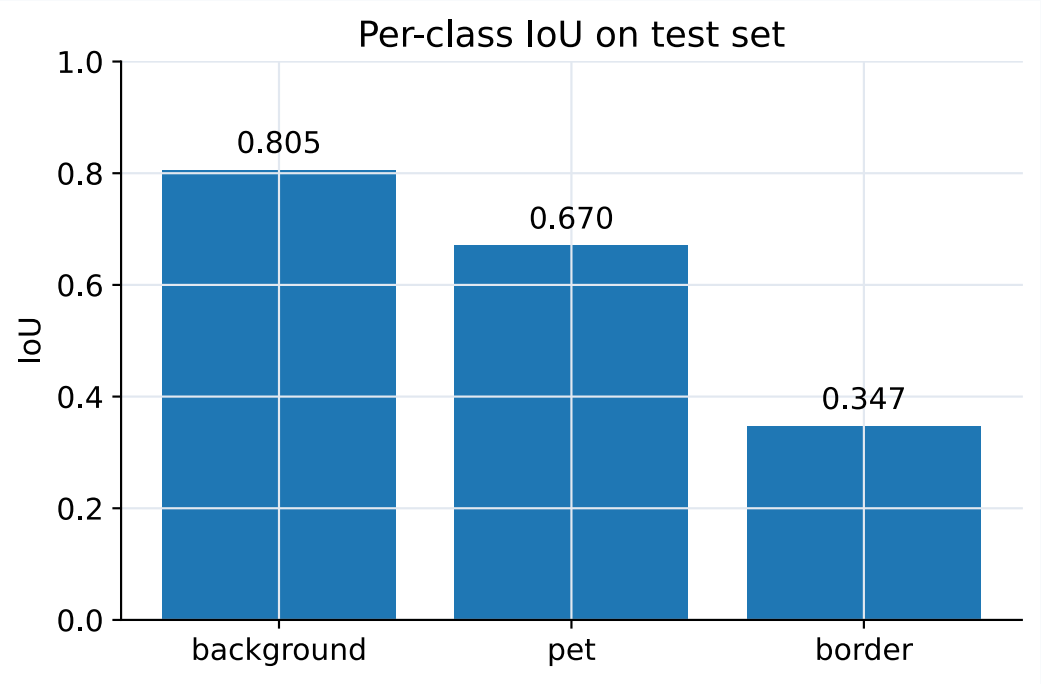
Mean IoU = average over classes

MeanIoU is more informative than pixel accuracy when background dominates.

Per-class IoU — What the Model Actually Learns

Section 02

The average is useful, but per-class values tell us where the model struggles.



Interpretation

- Background is easiest and largest.
- Pet region is learned reasonably well.
- Border is thin and ambiguous, so IoU is much lower.

Dice vs IoU — Two Ways to Measure Overlap

Section 02

Both reward overlap; Dice is often slightly more forgiving for small objects.

Intersection over Union

$|A \cap B| / |A \cup B|$

Penalizes false positives and false negatives.

Common in detection and segmentation.

Dice coefficient

$2|A \cap B| / (|A| + |B|)$

Equivalent to F1 for masks.

Popular in medical segmentation.

Practical rule

Report IoU or Dice with qualitative masks. Accuracy alone is not enough for segmentation.

In this lecture, the code focuses on MeanIoU because Keras provides it directly.

SECTION 03

FCN and U-Net

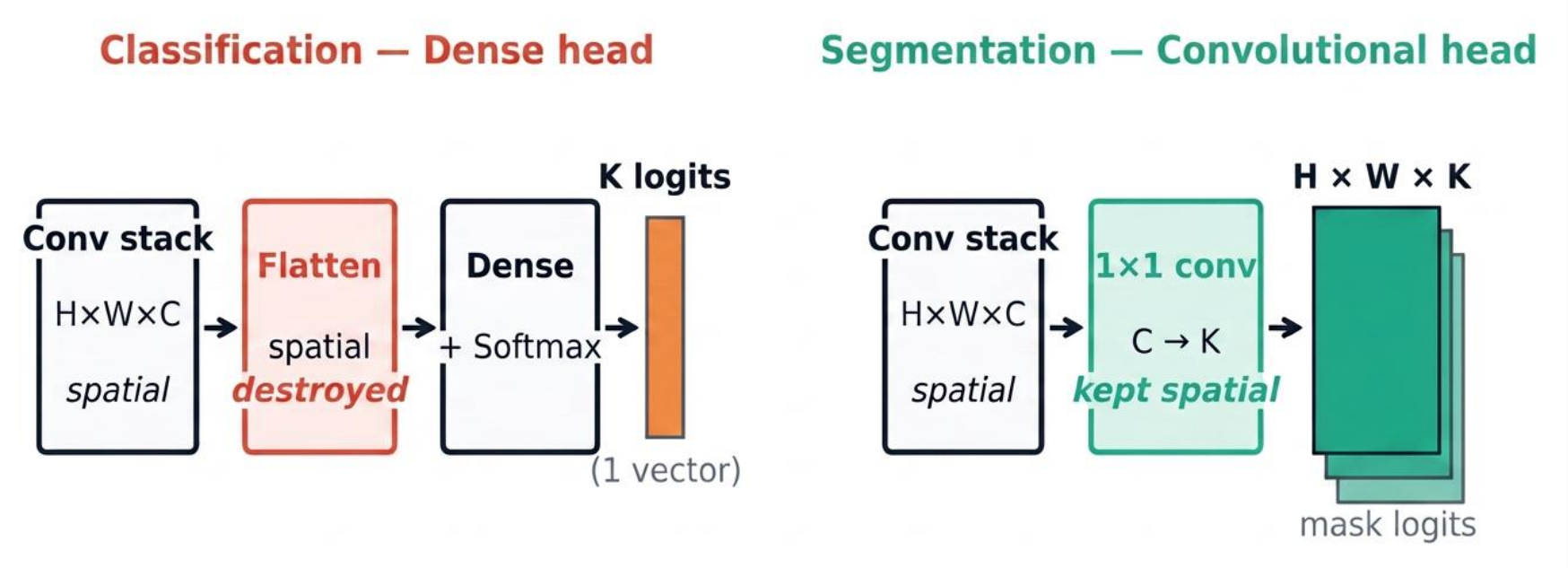
03

Encoder, decoder, and skip connections.

Why Dense Layers Are Not Enough

Section 03

Flatten + Dense throws away the spatial layout we need.



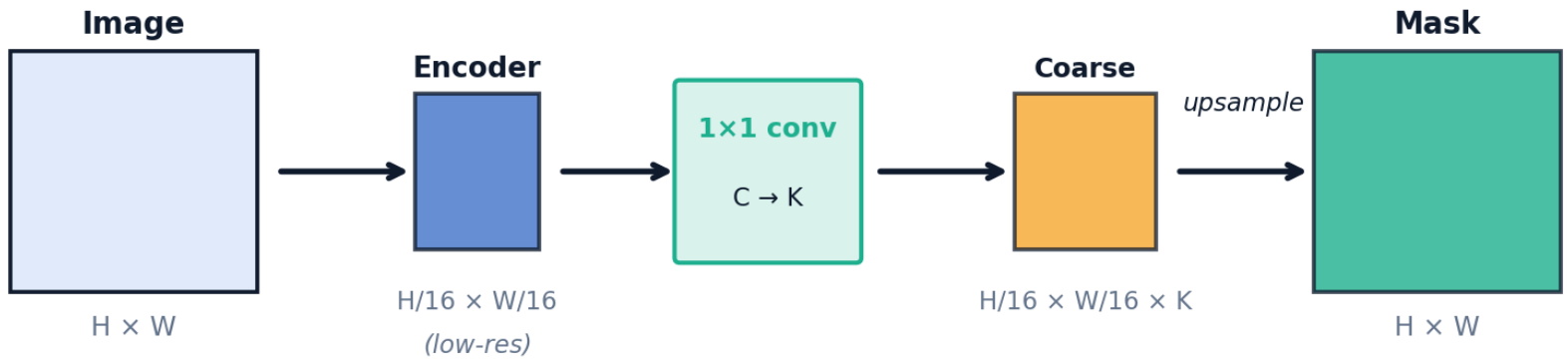
Take-away — Segmentation needs a spatial output, so the head must remain spatial: **no Flatten, no Dense.**

Same backbone CNN — the only change is replacing the classifier head with conv layers

Fully Convolutional Networks (FCN)

Section 03

Long, Shelhamer, & Darrell, CVPR 2015 — the foundation of modern segmentation.



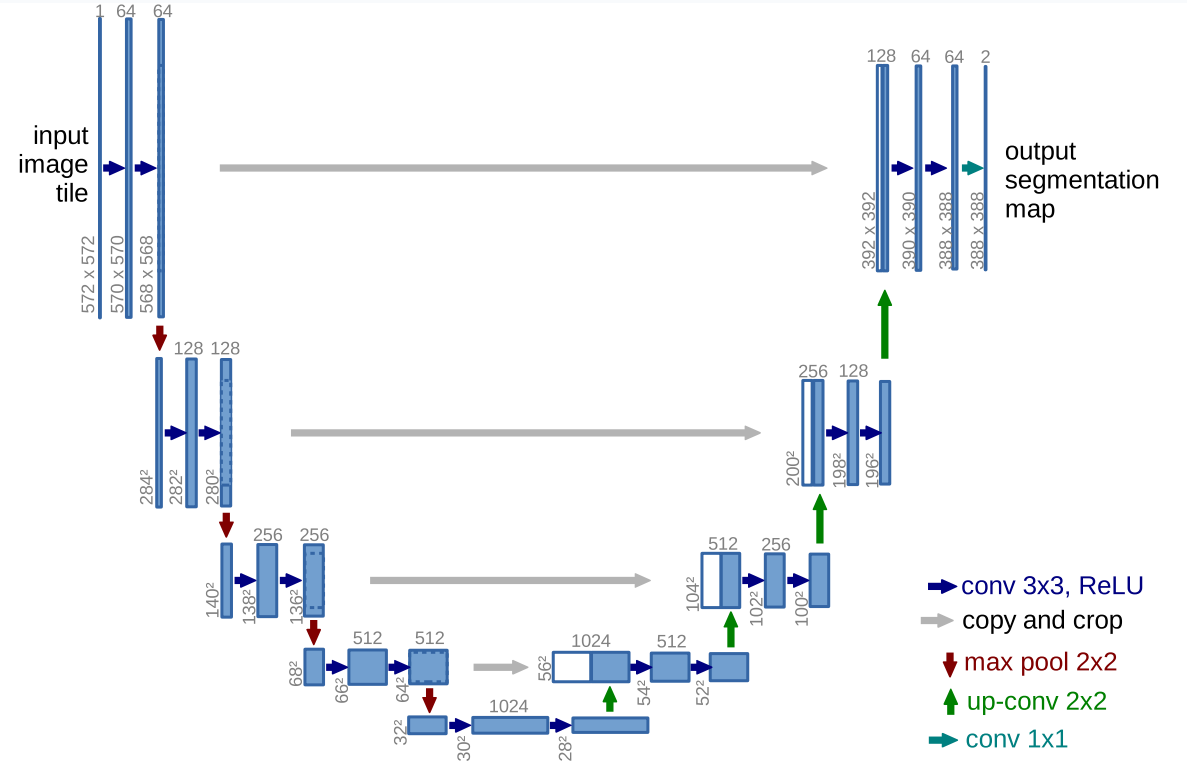
1. Convolutional all the way: an existing classifier (e.g. VGG) can be converted by replacing dense layers with 1×1 convs.
2. The encoder still downsamples — the output is spatial but coarse.
3. Naïve upsampling gives blocky masks. Better recovery comes from skip connections — the road that leads to U-Net.

FCN-32s, FCN-16s, FCN-8s — names refer to the upsampling factor of the final output

U-Net — The Big Picture

Section 03

Encoder + decoder + skip connections. Ronneberger et al., MICCAI 2015.



Encoder

Conv → Conv → MaxPool. Spatial size shrinks; channels grow.

Bottleneck

Lowest resolution, highest channel count — captures global context.

Decoder

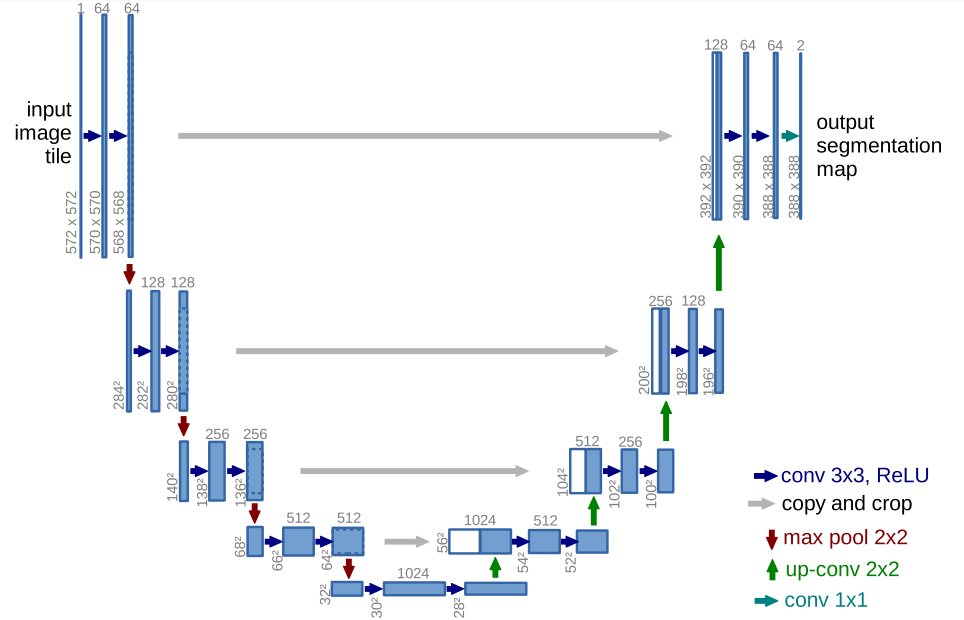
Up-conv + skip concat + Conv → Conv. Spatial size grows back.

The shape of the network gives U-Net its name; arrows indicate operations (see legend). Fig.1 from Ronneberger et al. (2015).

Encoder — The Contracting Path

Section 03

Build context at the cost of spatial resolution.



Repeated block

- Conv 3×3 + ReLU
- Conv 3×3 + ReLU
- MaxPool 2×2 (stride 2)

Each downsample step

- halves the spatial dimensions (H ↓, W ↓)
- doubles the channel count (C ↑)
- enlarges the receptive field

Why? Each MaxPool lets later filters "see" a larger area of the image, which is needed to recognize objects, not just textures.

Encoder = standard CNN classifier backbone; we keep it intact

Bottleneck — Context at Low Resolution

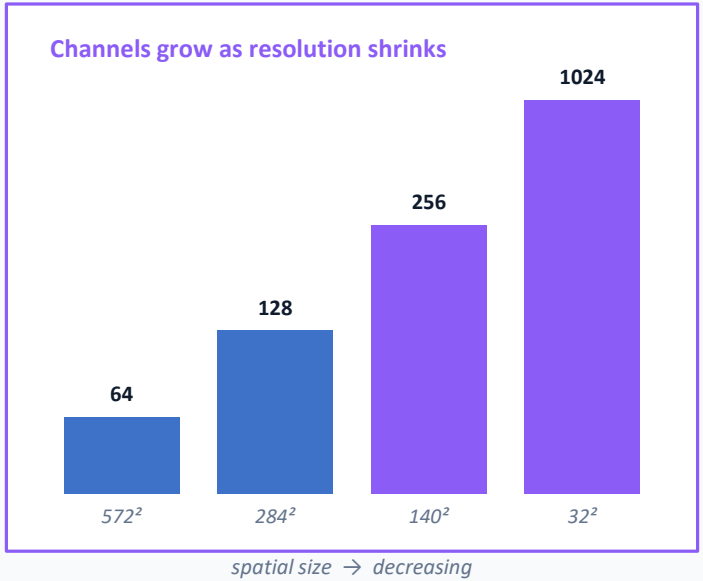
Section 03

The deepest features have the most channels and the smallest spatial extent.

At the bottom of the U:

- Spatial size is at its minimum (e.g. 32×32 for a 572×572 input).
- Channel count is at its maximum — 1024 in the original U-Net.
- Each spatial position now carries a long, abstract feature vector.
- Two more 3×3 convs combine these features without changing resolution.

→ This is where the network knows what is in the image. It now has to find out where.

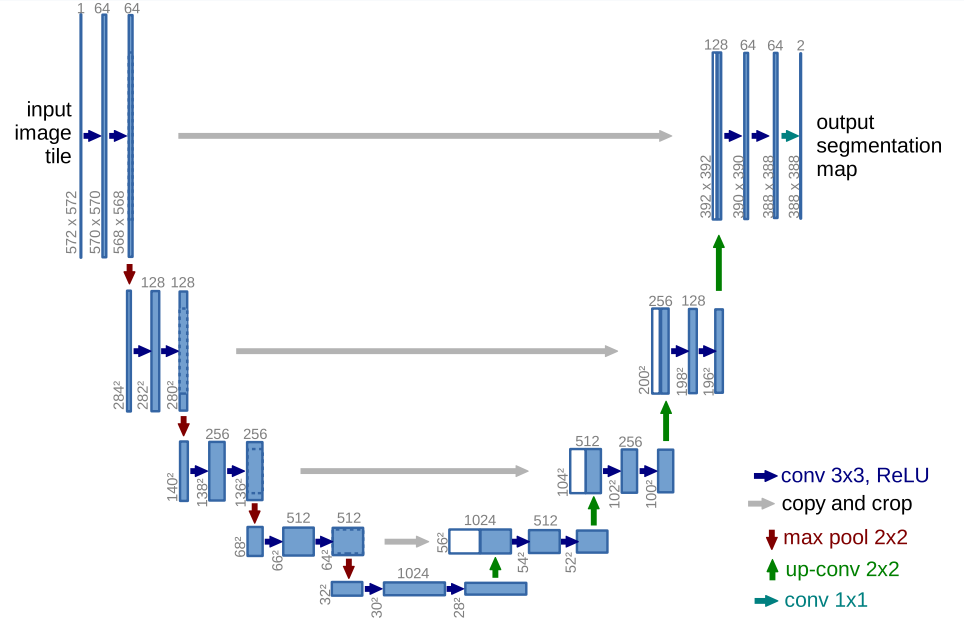


Channel count is the network's "vocabulary" — at the bottleneck it is the richest

Decoder — The Expanding Path

Section 03

Recover spatial resolution by combining context with skip features.



Repeated block

- Up-conv 2×2 (transposed conv, stride 2)
- Concatenate with the matching encoder feature map
- Conv 3×3 + ReLU
- Conv 3×3 + ReLU

Each upsample step

- doubles the spatial dimensions (H ↑, W ↑)
- halves the channel count (C ↓)
- brings back fine-grained boundary detail

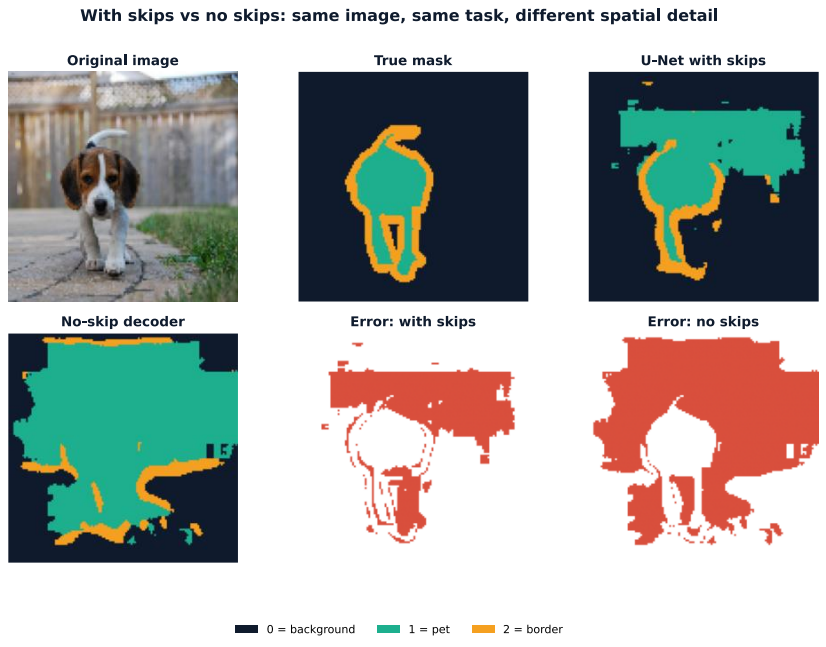
Final 1×1 conv — maps each pixel's 64-D feature vector to K class logits, ready for softmax.

Up-conv learns its upsampling weights, unlike fixed bilinear interpolation

Skip Connections — Preserving Spatial Detail

Section 03

Encoder features remember WHERE things are; decoder features know WHAT they are.



The encoder knows what is in the image; the skip connections help the decoder remember where it is.

Without skips: blocky, low-resolution masks. With skips: sharp, accurate boundaries

Quick Quiz — Match the U-Net Component

Section 03

For each behavior, name the U-Net component: encoder, decoder, or skip connection.

01

Halves the spatial size at each step, doubles the channel count — captures larger context as you go down.

?

02

Doubles the spatial size at each step using transposed convolutions — restores the original H×W resolution.

?

03

Wires earlier feature maps directly into later ones via concatenation — re-injects fine boundary detail.

?

Discuss with your neighbor — answer on the next slide

Answer — Match the U-Net Component

Section 03

Encoder → context; decoder → resolution; skip connections → fine boundary detail.

01

Halves the spatial size at each step, doubles the channel count — captures larger context as you go down.

→ **Encoder**

02

Doubles the spatial size at each step using transposed convolutions — restores the original H×W resolution.

→ **Decoder**

03

Wires earlier feature maps directly into later ones via concatenation — re-injects fine boundary detail.

→ **Skip connection**

Drop any one of these and U-Net stops being U-Net — each plays a distinct role.

Ablation Question: Skip Connections

Section 03

Do we really need them? Test the architecture idea by removing one component.

Hypothesis

Without skip connections, the decoder only sees compressed bottleneck features.

Expected result

Coarser masks, weaker boundaries, and lower border IoU.

Why this matters

- Ablation means removing or changing one design choice to measure what it contributes.
- It turns architecture diagrams into testable claims.

No-Skip U-Net Variant

Same encoder and decoder, but no concatenate operations from the encoder.

What is missing?

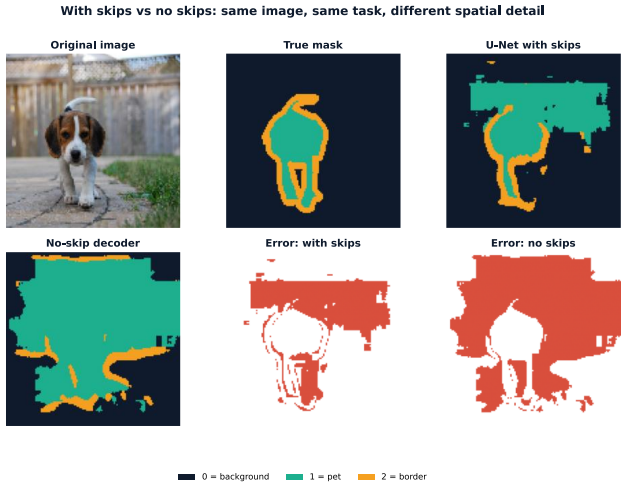
No high-resolution encoder features are passed to the decoder — only bottleneck information remains.

Compare this with U-Net: skip concatenations re-inject fine spatial detail into the decoder.

With Skips vs No Skips

Section 03

Skip connections preserve boundary detail that the bottleneck alone tends to lose.



Model	Test MeanIoU	Visual effect
U-Net with skips	≈0.61	sharper boundaries
No-skip decoder	≈0.58	weaker/coarser boundaries

In this short ablation, removing skip connections lowers MeanIoU slightly and tends to make masks less spatially precise.

Original U-Net vs Our Colab-Friendly U-Net

Section 03

We keep the idea — encoder, decoder, skip connections — at a smaller scale.

Property	Original U-Net (2015)	Our Colab U-Net
Depth (down/up)	4 down + 4 up	2 down + 2 up
Channels	64 → 128 → 256 → 512 → 1024	32 → 64 → 128 at the bottleneck
Padding	valid (output < input)	same (output = input)
Cropping	required at every concat	not needed
Output classes	2 (cell / background)	3 (background / pet / border)
Training cost	GPU, ~10 hrs originally	Colab T4, ~4–5 min

Same idea, smaller scale — encoder, decoder, and skip connections still do all the heavy lifting.

Small models run fast in Colab without losing the educational value of the architecture

Training U-Net with Little Data

Section 03

Augmentation and weighted loss — how U-Net wins from very few annotated images.

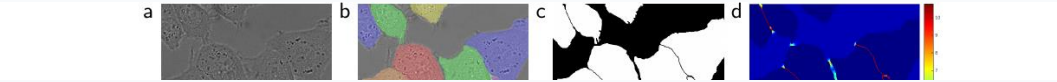


Fig. 3 — (a) cells, (b) instance overlay, (c) GT mask, (d) per-pixel weight map

The weight map (d) gives **high weight** to pixels in the narrow gaps between touching cells.

This forces the loss to focus on boundaries — a clever trick when only ~30 annotated images are available.

Heavy data augmentation

Random rotations, shifts, and elastic deformations multiply each annotated image many-fold.

Elastic deformation

Smooth random warps simulate natural shape variation — vital for medical/biological images.

Per-pixel weight map

Pixels on narrow gaps between touching objects get high weight; the loss is forced to learn boundaries.

U-Net's original setting: only ~30 annotated cell images. Our pet dataset is much larger, so we lean less on these tricks — but augmentation still helps.

Take-away: augmentation is the cheapest accuracy boost in segmentation

SECTION 04

Keras demo

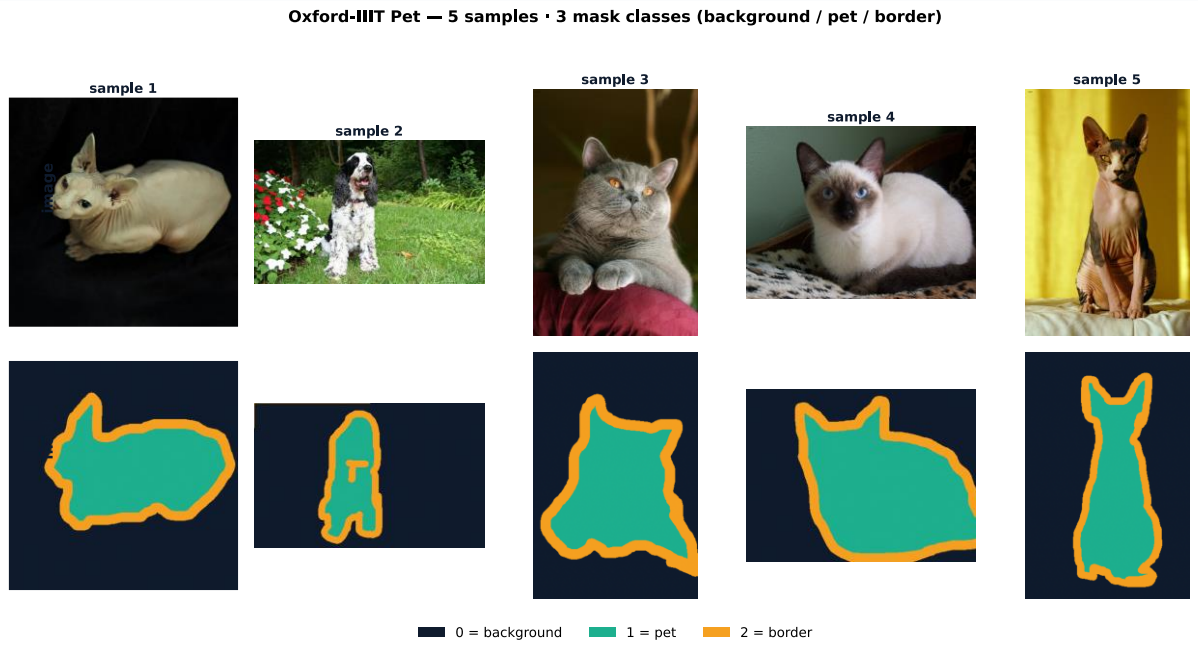
04

U-Net on Oxford-IIIT Pet, with metrics.

Dataset — Oxford-IIIT Pet Segmentation

Section 04

37 breeds, ~7,400 images. We use only the segmentation masks today.



~7,400 images

37 breeds of cat and dog, plus their masks.

3 mask classes

0 = background, 1 = pet, 2 = border.

Source

tensorflow_datasets ("oxford_iiit_pet").

Why use it?

Real photos, simple labels, fits in Colab memory.

Parkhi et al., 2012 — "Cats and Dogs". Dataset: tensorflow_datasets

Code — Preprocessing Images and Masks

Section 04

Resize image bilinearly; resize mask with nearest-neighbor to keep integer labels.

```python
IMG_SIZE    = 128
BATCH_SIZE  = 32
NUM_CLASSES = 3

def preprocess(example):
    img = tf.image.resize(example["image"], (IMG_SIZE, IMG_SIZE))
    img = tf.cast(img, tf.float32) / 255.0
    raw_mask = tf.squeeze(example["segmentation_mask"])

    # TFDS:    1=pet, 2=background, 3=border
    # Lecture: 0=background, 1=pet, 2=border
    mask = tf.where(
        tf.equal(raw_mask, 2), 0,
        tf.where(tf.equal(raw_mask, 1), 1, 2)
    )
    mask = tf.expand_dims(mask, -1)
    mask = tf.image.resize(mask, (IMG_SIZE, IMG_SIZE), method="nearest")
    mask = tf.cast(mask, tf.int32)
    return img, tf.squeeze(mask, -1)

def make_pipeline(ds, shuffle=False):
    if shuffle: ds = ds.shuffle(1024, seed=SEED)
    ds = ds.map(preprocess, num_parallel_calls=tf.data.AUTOTUNE)
    return ds.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

ds_train = make_pipeline(raw_train, shuffle=True)
```

method="nearest"

Mask values are class IDs — bilinear would invent fractional classes.

Normalize images

Divide by 255 → [0, 1]; matches the random init scale of U-Net.

Remap labels

TFDS uses 1=pet, 2=background, 3=border; lecture uses 0=background, 1=pet, 2=border.

Pipeline tuning

shuffle → batch → prefetch keeps the GPU fed during training.

Notebook §2 · IMG_SIZE matches the U-Net so all encoder-decoder shapes align

Code — Building a Small U-Net in Keras

Section 04

Two encoder stages + bottleneck + two decoder stages with skip concat.

```
def conv_block(x, filters):
    x = layers.Conv2D(filters, 3, padding="same", activation="relu")(x)
    x = layers.Conv2D(filters, 3, padding="same", activation="relu")(x)
    return x

def build_unet(img_size=128, n_classes=3):
    inputs = layers.Input(shape=(img_size, img_size, 3))
    c1 = conv_block(inputs, 32);  p1 = layers.MaxPool2D()(c1)   # encoder
    c2 = conv_block(p1, 64);      p2 = layers.MaxPool2D()(c2)
    bn = conv_block(p2, 128)                                    # bottleneck
    u2 = layers.Conv2DTranspose(64, 2, strides=2, padding="same")(bn)
    u2 = layers.Concatenate()([u2, c2]);  c3 = conv_block(u2, 64)   # decoder
    u1 = layers.Conv2DTranspose(32, 2, strides=2, padding="same")(c3)
    u1 = layers.Concatenate()([u1, c1]);  c4 = conv_block(u1, 32)
    outputs = layers.Conv2D(n_classes, 1, activation="softmax")(c4)
    return tf.keras.Model(inputs, outputs)

model = build_unet()
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=[
                  "accuracy",
                  tf.keras.metrics.MeanIoU(
                      num_classes=3,
                      sparse_y_pred=False,
                      name="mean_iou"
                  )
              ],
)
```

conv_block

Two 3×3 convs with same padding — keep the feature map shape.

Skip concatenate

Concatenate matching encoder + decoder features along channels.

1×1 conv head

n_classes filters → softmax over class axis at every pixel.

MeanIoU metric

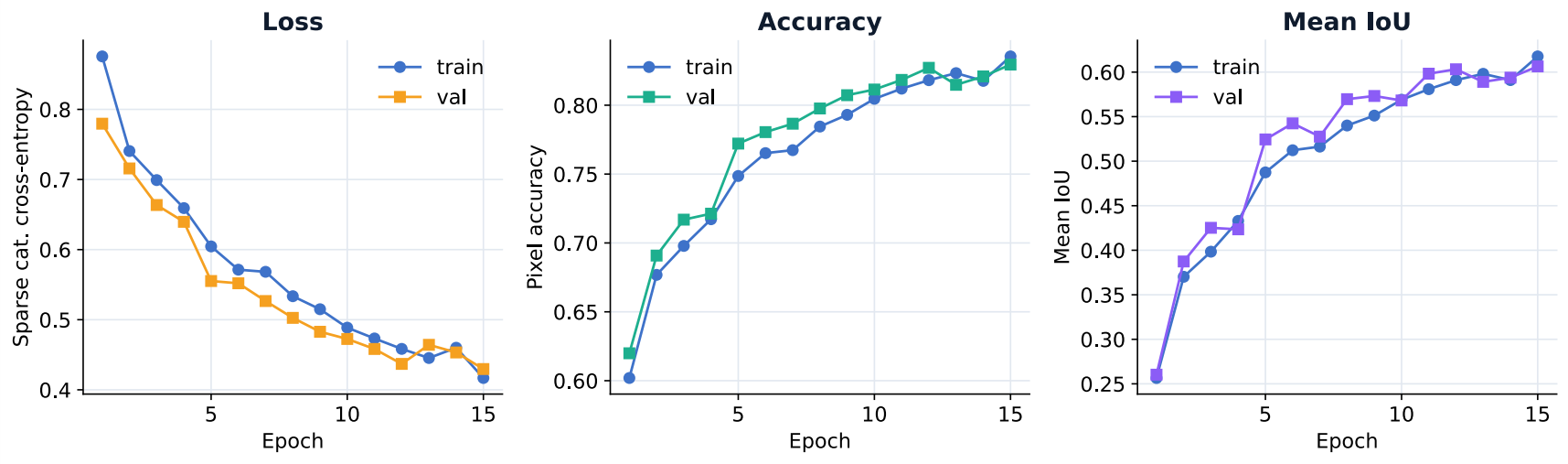
Always include — accuracy alone hides bad mask quality.

Notebook §3 · ~467K parameters total; trains in ~4–5 min on a Colab T4

Training Curves and Segmentation Metrics

Section 04

Loss falls steadily; mean IoU reaches ~0.60–0.61 in this Colab T4 run.



Loss

Train 0.417 / Val 0.430 at epoch 15 — no clear overfitting in this run.

Mean IoU

Train 0.606 / Val 0.609 — accuracy looks higher (0.83), but IoU gives a stricter view of mask quality.

When to stop

EarlyStopping on val_loss; restore_best_weights=True.

Colab T4 run · 15 epochs · training time 4.1 min · per-class IoU: bg 0.805 / pet 0.669 / border 0.353

Qualitative Results and Error Analysis

Section 04

Image · true mask · predicted mask · error map — boundaries are where errors live.

What the error map shows

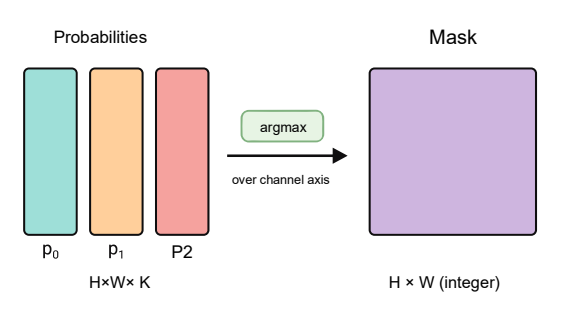
- Many errors occur around object boundaries — that is where labels are themselves uncertain.
- The small U-Net can also miss thin parts, produce false positives, or struggle with low contrast.
- Per-class IoU confirms this: border 0.35 << pet 0.67 << background 0.81.

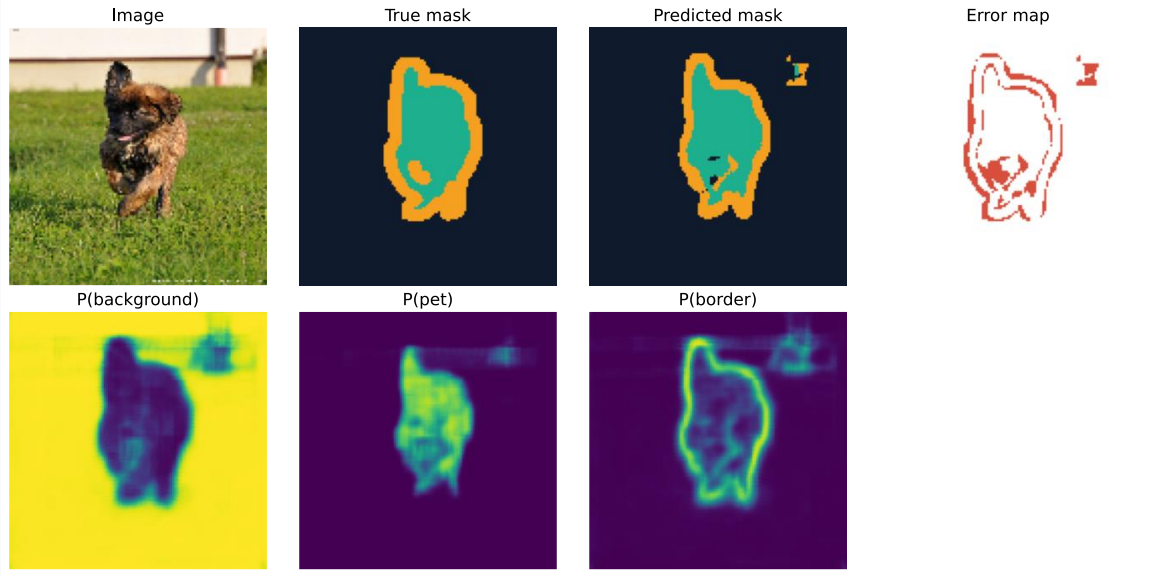
Always inspect a few predictions visually — metrics alone hide the failure modes

Softmax Probability Maps Before Argmax

Section 04

The model outputs probabilities first; the final mask is the argmax over classes.





Each pixel = a tiny classifier

- output K logits
- softmax → probabilities
- argmax → predicted class

Segmentation Augmentation

Section 04

Move image and mask together; photometric transforms affect the image only.

Classification

Transform image only

Label stays the same

Segmentation

Transform image and mask

Spatial alignment must remain exact

Bug to avoid

Flip/crop image but not mask

Training target becomes wrong

Example

- Horizontal flip: apply to both image and mask.
- Brightness or contrast: apply to image only; mask labels must not change.

Safe Augmentations for Oxford Pet

Section 04

Start simple: use transforms that preserve labels and alignment.

Horizontal flip

Image: yes

Mask: yes

Brightness

Image: yes

Mask: no

Contrast

Image: yes

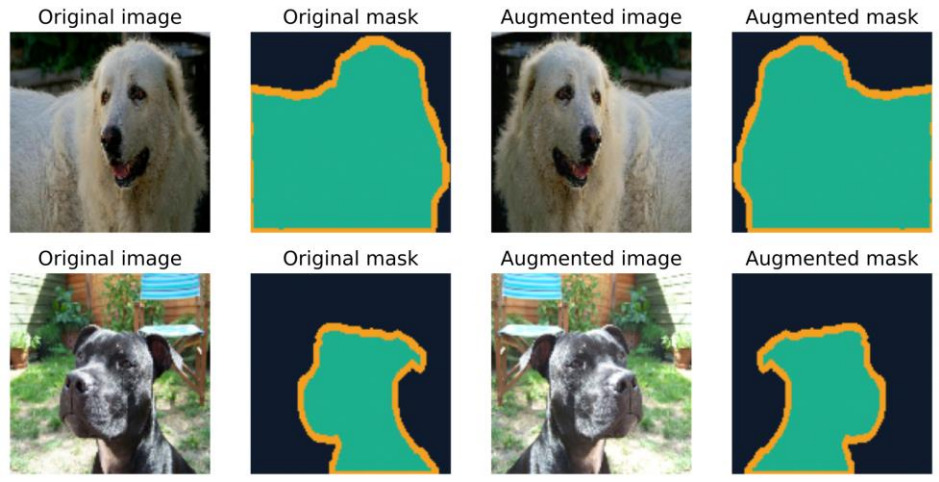
Mask: no

Crop / rotate

Image: yes

Mask: yes

Use care



Horizontal flip applied to BOTH image and mask — spatial alignment preserved.

SECTION 05

Beyond U-Net

05

Instance masks, YOLO-seg, and SAM.

From Semantic to Instance Segmentation

Section 05

When you need to count and track objects, semantic isn't enough.

Original

YOLO-seg — instance masks

Semantic

One mask per class — "all cats", no IDs.

Instance

One mask per object — separable IDs.

Mask R-CNN / YOLO-seg

Detection + per-box mask prediction.

YOLO-seg = real-time instance segmentation; Mask R-CNN = two-stage, more accurate

Promptable Segmentation with SAM

Section 05

Segment Anything Model — pretrained on 1B+ masks, controllable at inference time.

Box prompt

SAM mask

Point prompt

Click on an object; SAM returns its mask.

Box prompt

Draw a box; SAM segments the object inside.

Zero-shot

No retraining for new domains or classes.

SAM (Kirillov et al. 2023) shifts segmentation from "train per task" to "prompt at inference" — a foundation-model paradigm.

SAM 2 (2024) extends prompts to video segmentation with temporal consistency

Summary · Lectures 11-12

Wrap-up

Segmentation turns image understanding into pixel-level prediction.

01 **Segmentation = pixel-level classification**
Output is a H × W map of class IDs, not one label or a few boxes.

02 **Masks are structured outputs**
Integer images with 0 = background, 1+ = classes; visualized with a color map.

03 **U-Net = encoder + decoder + skip connections**
Encoder gives context; decoder restores resolution; skips preserve detail.

04 **Use IoU, not just accuracy**
Pixel accuracy can be dominated by background; IoU measures real overlap.

05 **Modern: instance and promptable masks**
YOLO-seg / Mask R-CNN for instances, SAM for zero-shot promptable masks.

Lecture 13 preview — Next: RNNs and Time-Series Forecasting.